

Diffusion: Implementing a Diffusion Transformer for Image Generation

Bhuvanesh Reddy

June 9, 2026

Abstract. This report describes my implementation of a diffusion transformer for image generation, with additional modules for Global Cyclic Refinement and Regionally-Adaptive Channel Denoising (RACD). I use Global Cyclic Refinement to encourage consistency across denoising stages and RACD to adaptively preserve fine details while reducing artifacts. I evaluate the model using CLIP-based Conditional Maximum Mean Discrepancy (CMMD) for semantic alignment and Fréchet Inception Distance (FID) for image quality.

1. Introduction.

Diffusion models have emerged as one of the most effective families of generative models for high-fidelity image synthesis. Unlike GANs, which learn a direct mapping from noise to images, diffusion models gradually transform noise into data through a sequence of stochastic denoising steps. This iterative refinement process allows diffusion models to capture complex data distributions and generate high-quality samples with lesser artifacts.

In this report, I implement a DiT (Diffusion Transformer) architecture, which leverages the self-attention mechanism of transformers to model long-range dependencies in images. I also incorporate two key modules: Global Cyclic Refinement, which promotes consistency across denoising stages by feeding intermediate outputs back into the model, and Regionally-Adaptive Channel Denoising (RACD), which adaptively preserves fine details while reducing noise.

2. Latent Diffusion Transformer Architecture. (DiT)

The DiT works not in the pixel space but in a latent space obtained from an autoencoder. This allows the model to focus on high-level features and also reduces the computational burden. It gets two inputs, one is the noisy latents and the other is the conditioning information (class labels and timestep embeddings).

2.1 Forward Diffusion Process. (Noising of the data)

The forward diffusion process gradually adds noise to the data over a series of timesteps. At each timestep t , the data x_0 is corrupted to produce a noisy version x_t . The noise is typically Gaussian and can be written as,

$$x_t = \sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon$$

Where α_t is a noise schedule that controls the amount of noise added at each timestep, and ϵ is sampled from a standard normal distribution. As t increases, the data becomes more corrupted, eventually converging to pure noise.

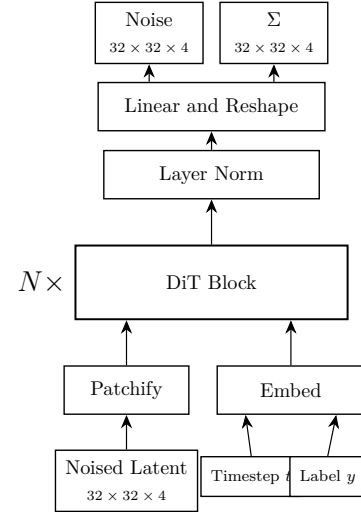


Figure 1. Latent Diffusion Transformer (DiT).

2.2 Class Labels and TimeStep Embeddings.

For this assignment, the dataset is only of Landscapes, so the class labels are unconditional, resulting in a single class.

The timestep embeddings however are generated using a sinusoidal positional encoding scheme, similar to the one used in the original Transformer architecture. This allows the model to incorporate information about the current timestep t into its computations, which is crucial for the denoising process. The timestep embedding can be defined as:

$$\text{TE}(t, 2i) = \sin\left(\frac{t}{10000^{2i/d}}\right), \quad \text{TE}(t, 2i+1) = \cos\left(\frac{t}{10000^{2i/d}}\right)$$

Where d is the dimensionality of the embedding space, and i is the index of the dimension. On top of this, there's an MLP that processes the timestep embeddings before they are concatenated with the class embeddings and fed into the transformer blocks.

$$\text{MLP}_{\text{TE}(t)} = W_2 \cdot \text{SiLU}(W_1 \cdot \text{TE}(t) + b_1) + b_2$$

2.3 Patchification and Position Embeddings.

The input image is first passed through a patchification process, where it is divided into patches of size 8×8 . Each patch is then flattened and projected into a higher-dimensional space using a linear layer, resulting in a sequence of patch embeddings, where each patch can be thought of as a *token* in the transformer architecture. To retain spatial information, learnable position embeddings are added to the path embeddings.

2.4 DiT Block.

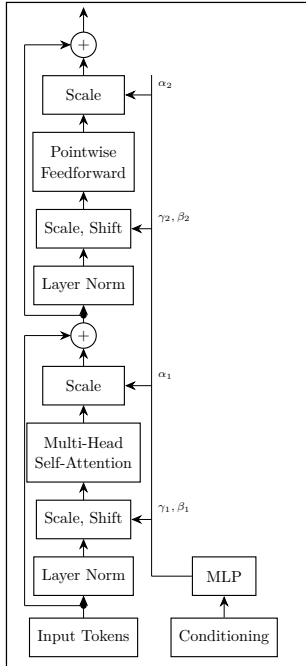


Figure 2. DiT Block with adaLN-Zero conditioning. The MLP processes the conditioning signal and predicts six modulation parameters $(\gamma_1, \beta_1, \alpha_1, \gamma_2, \beta_2, \alpha_2)$ applied to the attention and feedforward sublayers via scale/shift and residual scaling.

This is the core of the architecture, where the actual denoising happens. Each DiT block consists of a multi-headed self-attention mechanism, very similar to the one used in standard transformers. There are a few different ways in which conditioning can be applied in the DiT block, but in this implementation, I stick to the adaLN-Zero conditioning method, where the timestep and class embeddings are used to modulate the layer normalization parameters.

2.4.1 adaLN-Zero.

In the adaLN-Zero method, there are 6 parameters that are learned from the conditioning input.

$$X_c = X_{ts} + X_{cls}, \quad X_c \in \mathbb{R}^d$$

$$\mathcal{A} = \text{SiLU}(W_a X_c + b_a), \quad \mathcal{A} \in \mathbb{R}^{6d}$$

And then we split the $\mathcal{A}$ into 6 features which are used to modulate the layernorms.

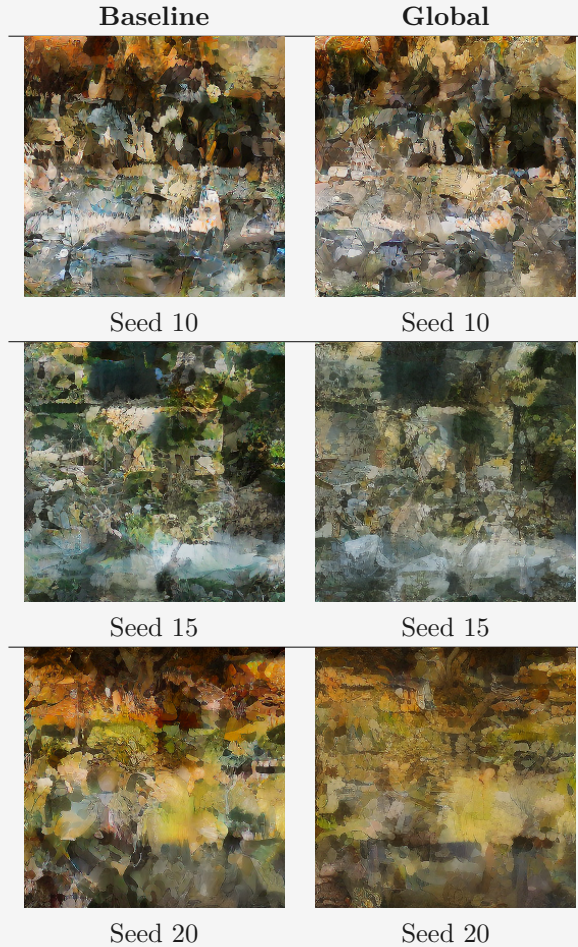
2.5 Reverse Diffusion Process. (Denoising)

For both forward and reverse processes, we use the frozen autoencoder from the original DiT paper to encode images into the latent space and decode them back. The reverse diffusion process is where the actual generation happens. After the DiT predicts a noise estimate $\hat{\epsilon}$, there'll be a step to compute the denoised latents $\hat{x}_0$ and the variance Σ . The model is trained to minimize the mean squared error between the predicted noise and the true noise added in the forward process.

3. Global Cyclic Refinement.

In this process, we take the predicted latents from the DiT and feed them back into the model for a second pass of denoising, as a *refinement* step. For this assignment, the timestep has been set to $t = 200$ for the refinement pass, which is a mid-way point in the diffusion process.

Remark: Is there a visual improvement over baseline?



As you can see, there's a marginal *difference* in the generated images. Whether it is better may be subjective. The objective thing is that the CMMD score is worse for the global cyclic refinement, which implies that the global cyclic refinement is actually making the generated images less semantically aligned with the reference images. In this case, the DiT is still undertrained, so the global refinement just kinda blurs out the image, reducing the already existing features.

4. Regionally Adaptive Channel Denoising (RACD).

RACD is like a middle ground between the two extremes of aggressive denoising (which can lead to loss of details) and conservative denoising (which can leave artifacts). The idea is to adaptively determine how much of the predicted noise to subtract from the input latents, based on the confidence of the model’s predictions. We first train a *Difficulty Predictor*, by freezing the DiT and training a linear layer on top of the features from the last attention layer. Using Tweedie’s formula, we estimate the latent prediction.

$$x_0^{\text{pred}} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \sqrt{1 - \alpha_t} \hat{\epsilon} \right)$$

The difference between the predicted latents and the ground truth latents is used as a measure of difficulty. During inference, the scores map is computed using the difficulty predictor, and a threshold τ is applied to determine which regions should be denoised more aggressively.

5. Evaluation Metrics.

For evaluation, I use two metrics: CLIP-based Conditional Maximum Mean Discrepancy (CMMD) and Fréchet Inception Distance (FID). It measures the distance between the distribution of reference (real) images and generated (fake) images using CLIP embeddings. It then applies the Maximum Mean Discrepancy (MMD) statistical test with a Gaussian RBF kernel. FID on the other hand, measures the distance between the feature distributions of real and generated images using a pre-trained Inception network. CLIP is more reliable in low sample number regimes, while FID is more sensitive to sample quality and diversity.

Table 1. Quantitative evaluation of fidelity and computational efficiency across baseline DiT, global cyclic refinement, and RACD variants. Generation time is for 200 samples.

Configuration	FID ↓	CMMD ↓	Gen Time (s)
Baseline DiT	305.20	4.76	5.50
Global Cyclic	292.90	4.96	6.40
RACD ($\tau = 0.3$)	301.11	4.80	6.50
RACD ($\tau = 0.5$)	302.40	4.81	6.50
RACD ($\tau = 0.7$)	296.04	4.81	6.50

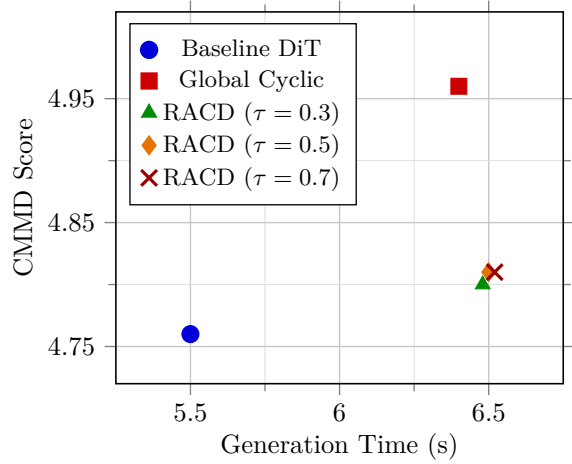


Figure 3. Generative fidelity (CMMD) vs. computational efficiency trade-off.

Remark: Compute Trade-Offs and why no effect of changing τ ?

The global cyclic refinement adds an extra pass through the DiT, which increases the generation time by about 0.9 seconds on average. RACD adds a small overhead due to the additional linear layer and the computation of the scores map, but this is relatively minor compared to the overall generation time.

As for the lack of significant difference in CMMD scores across different τ values, it could be that since the DiT is still undertrained the intermediate noise predictions are not yet fully calibrated, the predictor network struggles to make nuanced, continuous distinctions between high and low-difficulty patches. As a result, the scores map M may not have a strong correlation with actual patch difficulty, leading to similar denoising behavior across different τ thresholds and thus similar CMMD scores. Optimal tuning of τ requires a more mature DiT, but for

the sake of this assignment, i have notices in some testings that the benchmarks of $\tau = 0.3$ being better, but most of the times they are similar.

6. Configuration.

The DiT backbone shared by all three configurations was trained with the settings in Table 2. Training is unconditional (ε -prediction, MSE loss) on VAE latents of a landscape dataset; the same checkpoint (`dit_best.pt`) is used for baseline, Global Cyclic and RACD sampling.

Table 2. Training configuration for the DiT backbone.

Architecture (DiT)		Optimization	
Hidden size	768	Optimizer	AdamW
Transformer blocks	12	Learning rate	1×10^{-4}
Attention heads	12	Weight decay	1×10^{-4}
Patch size	8	Precision	Mixed (AMP)
Latent channels	4	Batch size	16×2 accum. (eff. 32)
Conditioning	Unconditional	Epochs	100
Diffusion / data			
Train timesteps	1000	VAE	SD-VAE-ft-ema (frozen)
Noise schedule	linear $\beta = 10^{-4} \rightarrow 0.02$	Latent resolution	$32 \times 32 \times 4$
Objective	ε -pred, MSE	Image resolution	256×256

7. Future Scope.

I believe that this DiT was severely undertrained, due to which a lot of the nuances of the refinement schemes were not fully realized. With more training, I expect to see a more significant improvement from the global cyclic refinement, and a more noticeable difference between different τ values in RACD.

8. Generations.

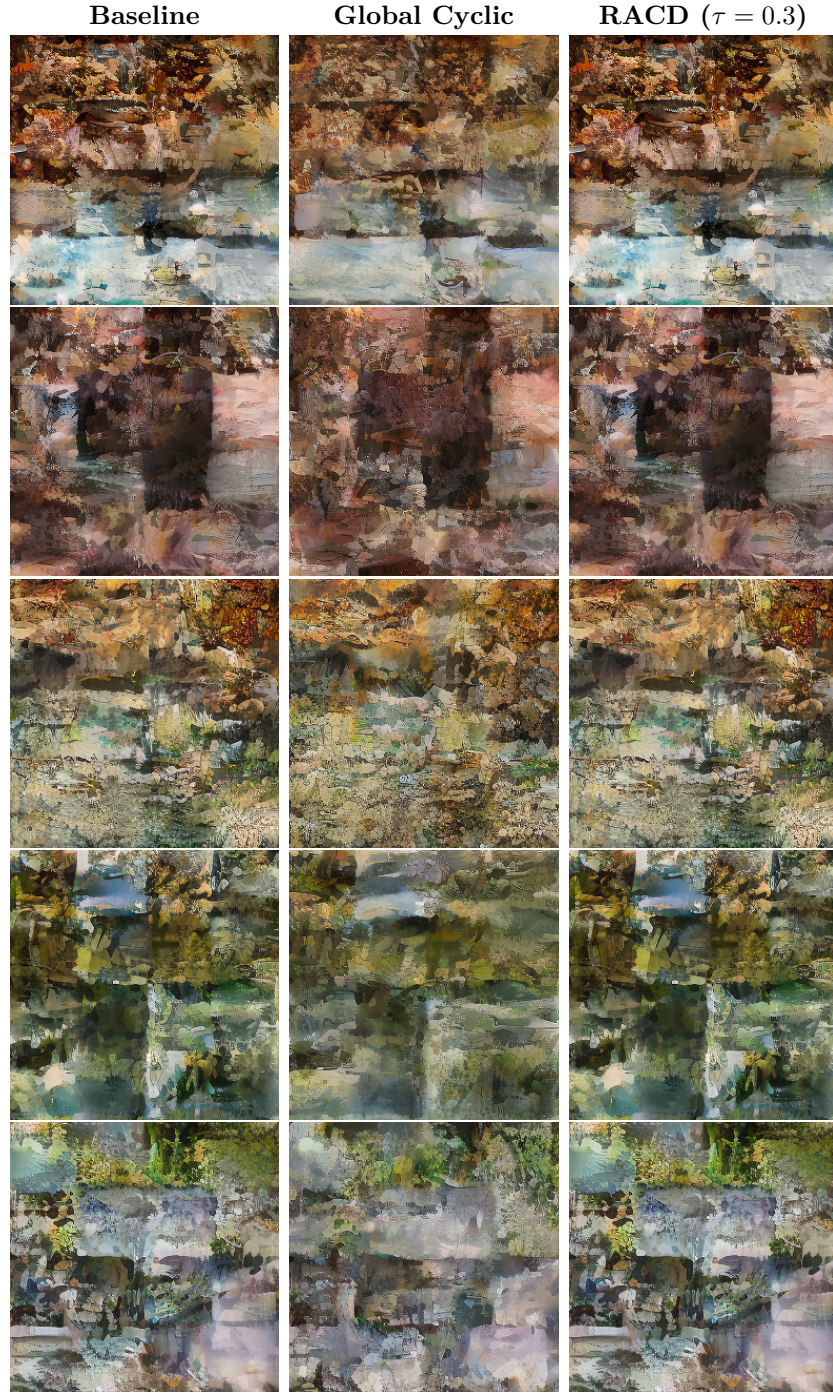


Figure 4. Qualitative comparison across configurations. Each row shares a single random seed across the three columns, so differences are attributable to the refinement scheme rather than the initial noise. Sampling config: 50 DDIM steps over a $32 \times 32 \times 4$ latent (1000 train timesteps), decoded with the SD-VAE. Global Cyclic and RACD both re-inject noise up to timestep $t = 200$; RACD masks refinement to high-difficulty patches on a 4×4 grid with threshold $\tau = 0.3$.